Explain Multithreading and Thread Life Cycle with a Neat Diagram.

1. Introduction

Java supports **multithreaded programming**, which allows more than one task to execute concurrently within a program.

A thread is a **lightweight unit of execution**. Each thread represents an independent path of execution.

2. Definition of Multithreading

Multithreading is the process of executing multiple threads concurrently within a single program. A Java program can contain multiple threads that share the resources of the same process.

Example

In a web browser:

- One thread downloads a file.
- Another thread displays the webpage.
- Another thread handles user input.

Thus, multiple tasks can be performed concurrently.

3. Advantages of Multithreading

1. **Better CPU utilization** – CPU can perform another task when one thread is waiting.
2. **Concurrent execution** – Multiple tasks can progress concurrently.
3. **Improved responsiveness** – The application can continue responding to the user.
4. **Efficient resource sharing** – Threads share resources of the same process.
5. **Faster execution** – Independent tasks can execute concurrently, particularly on multicore systems.

4. Creating a Thread

A thread can be created by extending the Thread class.

Syntax

```
class MyThread extends Thread
{
   public void run()
   {
      // thread task
   }
}
```

Example

```
class MyThread extends Thread
{
   public void run()
   {
      System.out.println("Thread is running");
   }

   public static void main(String args[])
   {
      MyThread t = new MyThread();
```

```
        t.start();
    }
}
```

Explanation

1. MyThread extends the Thread class.
2. The run() method contains the task.
3. An object t is created.
4. start() is called to begin thread execution.
5. The JVM schedules the thread for execution.

5. Thread Life Cycle

The **thread life cycle** represents the different states through which a thread passes during its execution.

The important states are:

1. New
2. Runnable
3. Running
4. Waiting/Blocked
5. Terminated

5.1 New State

A thread enters the **New state** when its object is created.

Example

Thread t = new Thread();

At this stage, the thread has been created but execution has not started. (before the start() method is called)

5.2 Runnable State

When start() is called, the thread becomes **Runnable**.

t.start();

The thread is ready for execution and waits for the thread scheduler to allocate CPU time.

Important: Runnable means **ready to run**, not necessarily running.

5.3 Running State

When the thread scheduler selects the thread, it starts executing the run() method.

```
public void run()
{
    System.out.println("Thread executing");
}
```

The thread is now in the **Running state**.

5.4 Waiting / Blocked State

A running thread may temporarily stop executing.

This may occur when:

- The thread is waiting for a lock.
- The thread calls wait().
- The thread calls sleep().

- The thread waits for a resource.
- The thread waits for another thread.

After the required condition is satisfied, it can become eligible to run again.

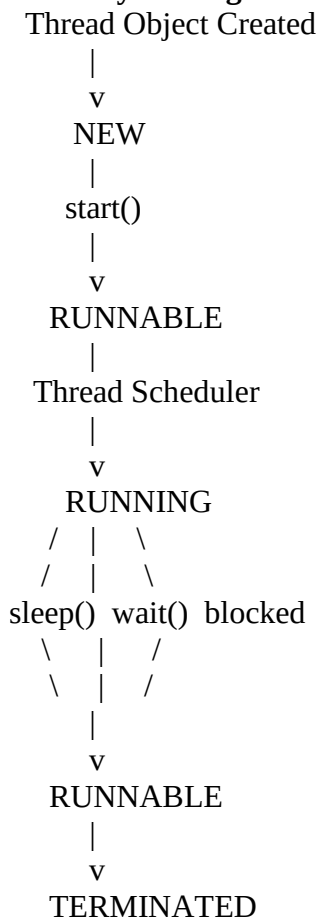
5.5 Terminated State Dead State

A thread enters the Dead (terminated) state when its run() method finishes execution normally, or when it is explicitly killed by calling stop(). Once a thread is dead, it cannot be restarted by calling start() again — doing so throws an IllegalThreadStateException.

When the run() method finishes its execution, the thread enters the **Terminated state**.

A terminated thread **cannot be restarted**.

6. Thread Life Cycle Diagram



7. Important Methods

Method	Purpose
start()	Starts a new thread
run()	Defines the task of a thread
sleep()	Temporarily pauses a thread
wait()	Makes a thread wait
join()	Waits for another thread to finish

8. Conclusion
Thus, **multithreading** enables concurrent execution of multiple tasks. The **thread life cycle** describes the movement of a thread from creation to execution, temporary waiting and finally termination.

---

2. Explain Thread Priority in Java with Suitable Example.
1. Introduction
When several threads are ready for execution, Java provides a mechanism to indicate their relative importance. This mechanism is called **thread priority**.

2. Definition
Thread priority is a value assigned to a thread that indicates its relative importance to the thread scheduler.
Java supports priority values from **1 to 10**.

3. Predefined Priorities
Java provides three predefined priority constants.

Constant	Value
Thread.MIN_PRIORITY	1
Thread.NORM_PRIORITY	5
Thread.MAX_PRIORITY	10

Important Point
The default or normal priority is:
Thread.NORM_PRIORITY
Its value is **5**.

4. setPriority() Method
The setPriority() method is used to assign a priority to a thread.
Syntax
threadObject.setPriority(value);
Example
t.setPriority(8);
Here, priority 8 is assigned to thread t.

5. getPriority() Method
The getPriority() method is used to obtain the priority of a thread.
Syntax
threadObject.getPriority();
Example
System.out.println(t.getPriority());

6. Example Program
```
class PriorityDemo extends Thread
{
   public void run()
```

```
    {
        System.out.println(
            Thread.currentThread().getName()
            + " Priority = "
            + Thread.currentThread().getPriority());
    }

    public static void main(String args[])
    {
        PriorityDemo t1 = new PriorityDemo();
        PriorityDemo t2 = new PriorityDemo();

        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.MAX_PRIORITY);

        t1.start();
        t2.start();
    }
}
```

7. Explanation of the Program

1. PriorityDemo extends the Thread class.
2. The run() method displays the thread name and priority.
3. Two thread objects, t1 and t2, are created.
4. t1 is assigned minimum priority.
5. t2 is assigned maximum priority.
6. Both threads are started using start().
7. Their priority values are displayed.

8. Important Points about Thread Priority

1. Java thread priority ranges from **1 to 10**.
2. Minimum priority is **1**.
3. Normal priority is **5**.
4. Maximum priority is **10**.
5. setPriority() is used to set priority.
6. getPriority() is used to obtain priority.
7. A newly created thread normally inherits the priority of its parent thread.
8. Higher priority gives a thread greater scheduling preference.
9. Priority does **not guarantee execution order**.
10. Actual scheduling depends on the JVM and operating system.

9. Conclusion

Thread priority helps the scheduler determine the **relative importance of threads**. However, programmers should not depend on priority to guarantee a particular execution order.

Q3. Explain Synchronization in Java with Suitable Examples.

1. Introduction

In multithreaded programming, multiple threads may access the same resource at the same time. If access is not properly controlled, it may produce incorrect or inconsistent results.
Java provides **synchronization** to solve this problem.

2. Definition

Synchronization is a mechanism used to control the access of multiple threads to shared resources.

3. Need for Synchronization

Synchronization is required to:

1. Prevent **race conditions**.
2. Protect shared resources.
3. Maintain data consistency.
4. Protect critical sections.
5. Control concurrent access.

4. Shared Resource

A resource accessed by more than one thread is called a **shared resource**.

Examples

- Common variable
- Array
- File
- Database record
- Bank account
- Object

5. Race Condition

A **race condition** occurs when multiple threads access and modify shared data concurrently and the final result depends on the timing or order of execution.

Example

Suppose:
count = 10
Two threads execute:
Thread 1 → count++
Thread 2 → count++
The expected result is:
12
But if both threads read the value 10 before either update is completed, one update may be lost and the result may become:
11
This is called a **race condition**.

6. Critical Section

The portion of code that accesses or modifies a shared resource is called a **critical section**.

Example

count++;
This statement can be treated as a critical section when multiple threads update the same count.

7. Object Lock
Synchronization in Java uses an **object monitor/lock**.
When a thread enters a synchronized method or block:

1. It obtains the required lock.
2. It executes the critical section.
3. It releases the lock.
4. Another waiting thread can obtain the lock.

8. Synchronized Method
A method can be declared using the synchronized keyword.
Syntax

```
synchronized void methodName()
{
   // critical section
}
```

Example

```
class Counter
{
   int count = 0;

   synchronized void increment()
   {
      count++;
   }
}
```

Explanation

- count is a shared variable.
- increment() modifies the shared variable.
- The method is declared synchronized.
- Only one thread at a time can execute the synchronized instance method for the same object.
- Therefore, access to count is controlled.

9. Synchronized Block
Sometimes the entire method need not be synchronized.
Only the critical section can be protected using a synchronized block.
Syntax

```
synchronized(object)
{
   // critical section
}
```

Example

```
class Counter
```

```
{
   int count = 0;

   void increment()
   {
      synchronized(this)
      {
         count++;
      }
   }
}
```

Explanation

1. The method itself is not synchronized.
2. Only count++ is protected.
3. this represents the current object.
4. The synchronized block protects the critical section.
5. This provides finer control than synchronizing the complete method.

10. Synchronized Method vs Synchronized Block

Synchronized Method	**Synchronized Block**
Entire method is protected	Only selected statements are protected
Simple to write	Provides finer control
More code may be locked	Only critical section is locked
Uses synchronized in method declaration	Uses synchronized(object)
Suitable when whole method needs protection	Suitable when only part needs protection

11. Complete Example

```
class Counter
{
   int count = 0;

   synchronized void increment()
   {
      count++;
   }
}

class MyThread extends Thread
{
   Counter c;

   MyThread(Counter c)
   {
      this.c = c;
```

```
    }

    public void run()
    {
        for(int i = 1; i <= 1000; i++)
        {
            c.increment();
        }
    }
}

class SyncDemo
{
    public static void main(String args[])
    {
        Counter c = new Counter();

        MyThread t1 = new MyThread(c);
        MyThread t2 = new MyThread(c);

        t1.start();
        t2.start();

        try
        {
            t1.join();
            t2.join();
        }
        catch(InterruptedException e)
        {
            System.out.println(e);
        }

        System.out.println("Count = " + c.count);
    }
}
```

Program Explanation

1. Counter contains the shared variable count.
2. increment() changes the value of count.
3. The method is synchronized.
4. Two threads use the **same Counter object**.
5. Both threads execute increment().
6. Synchronization controls access to count.
7. join() makes the main thread wait until both threads complete.
8. Finally, the value of count is displayed.

12. Advantages of Synchronization

1. Prevents race conditions.
2. Maintains data consistency.
3. Protects shared resources.
4. Provides controlled access to critical sections.
5. Makes multithreaded programs safer and more reliable.

13. Conclusion

Synchronization is an important feature of Java multithreading. It ensures that **shared resources are accessed in a controlled manner** and prevents inconsistent results caused by concurrent execution.

---

4. Explain Thread Exceptions in Java.

1. Introduction

During thread execution, abnormal conditions may occur. These conditions are called **exceptions**.

Java provides exception-handling mechanisms to handle such situations.

2. Definition

An **exception** is an abnormal condition that interrupts the normal flow of program execution. Thread-related operations can also generate exceptions.

3. Important Thread Exceptions

1. InterruptedException

This exception can occur when a thread that is sleeping or waiting is interrupted.

Example

```
try
{
   Thread.sleep(1000);
}
catch(InterruptedException e)
{
   System.out.println("Thread interrupted");
}
```

Explanation

- sleep(1000) pauses the thread for approximately 1000 milliseconds.
- If the thread is interrupted during the sleep, InterruptedException can occur.
- The exception is handled using catch.

4. IllegalThreadStateException

This exception occurs when a thread operation is performed when the thread is in an inappropriate state.

Example

```
class Demo extends Thread
{
   public void run()
   {
      System.out.println("Running");
   }

   public static void main(String args[])
   {
      Demo t = new Demo();

      t.start();
      t.start();
   }
}
```

Here, the same thread is started twice.

The second start() can result in:
IllegalThreadStateException

5. Exception Handling
Thread exceptions can be handled using try-catch.
Syntax

```
try
{
   // statements
}
catch(Exception e)
{
   // handling
}
```

6. Advantages of Exception Handling

1. Prevents abnormal program termination.
2. Helps identify errors.
3. Improves program reliability.
4. Provides controlled handling of exceptional conditions.
5. Makes programs easier to maintain.

7. Conclusion
Exception handling is essential in multithreaded programs because it allows abnormal thread conditions to be handled safely without unnecessarily terminating the complete application.

---

5. Explain Inter-Thread Communication using wait(), notify() and notifyAll().

1. Introduction

When multiple threads work together, they may need to communicate and coordinate their activities.

Java provides **inter-thread communication** mechanisms for this purpose.

2. Definition

Inter-thread communication is a mechanism through which threads communicate and coordinate with one another.

It is especially useful when one thread must wait for another thread to complete a particular operation.

3. Important Methods

Java provides:

1. wait()
2. notify()
3. notifyAll()

These methods are defined in the **Object class**.

4. wait()

The wait() method causes the current thread to wait until it is notified or otherwise resumes according to the waiting conditions.

Syntax

```
obj.wait();
```

Example

```
synchronized(obj)
{
   obj.wait();
}
```

Important Point

wait() releases the object's monitor while the thread is waiting.

5. notify()

The notify() method wakes up **one thread** waiting on the same object's monitor.

Syntax

```
obj.notify();
```

Example

```
synchronized(obj)
{
   obj.notify();
}
```

6. notifyAll()

The notifyAll() method wakes up **all threads** waiting on the same object's monitor.

Syntax

```
obj.notifyAll();
```

Example

```
synchronized(obj)
{
   obj.notifyAll();
}
```
The awakened threads then compete to acquire the lock.

7. Difference Between wait(), notify() and notifyAll()

Method	Function
wait()	Makes the current thread wait
notify()	Wakes one waiting thread
notifyAll()	Wakes all waiting threads

8. Example Concept – Producer and Consumer

A common application of inter-thread communication is the **Producer-Consumer problem**.

```
Producer Thread
     |
     v
 Produces Data
     |
     v
 Shared Resource
     |
     v
Consumer Thread
     |
     v
 Consumes Data
```

If the resource is empty, the consumer may wait.
If the producer produces data, it can notify the waiting consumer.
Thus, wait() and notification methods help coordinate the threads.

9. Important Points

1. wait(), notify() and notifyAll() belong to the Object class.
2. They are used for inter-thread communication.
3. They are normally called within synchronized code.
4. wait() makes a thread wait.
5. notify() wakes one waiting thread.
6. notifyAll() wakes all waiting threads.
7. These methods help coordinate multiple threads.

10. Conclusion

Inter-thread communication allows threads to **coordinate their activities efficiently**. The methods wait(), notify() and notifyAll() are important mechanisms for communication between cooperating threads.

---

6. Explain the Difference Between wait() and sleep().

This is a very useful **5/10-mark university question**.

wait()	**sleep()**
Defined in Object class	Defined in Thread class
Used mainly for inter-thread communication	Used to pause a thread for a specified time
Releases the object's monitor while waiting	Does not release an acquired monitor
Normally used with synchronization	Does not require synchronization merely to call it
Resumes after notification/interruption or other wait condition	Resumes after the specified sleep time or interruption

Example of sleep()

Thread.sleep(1000);

The current thread pauses for approximately 1000 milliseconds.

Example of wait()

```
synchronized(obj)
{
   obj.wait();
}
```

The thread waits until it is notified or otherwise resumes according to the waiting conditions.

Conclusion

sleep() is mainly used for **time-based delay**, whereas wait() is mainly used for **thread coordination and communication**.

---

7. Explain Synchronized Method and Synchronized Block.

Introduction

Synchronization is required when multiple threads access shared resources.

Java provides two common ways to synchronize code:

1. Synchronized method
2. Synchronized block

1. Synchronized Method

Syntax

```
synchronized void display()
{
   // statements
}
```

The complete method is protected.

Example

```
class Demo
{
   synchronized void display()
   {
      System.out.println("Hello");
   }
}
```

2. Synchronized Block

Syntax

```
synchronized(obj)
{
   // critical section
}
```

Example

```
class Demo
{
   void display()
   {
      synchronized(this)
      {
         System.out.println("Hello");
      }
   }
}
```

Difference

Synchronized Method	**Synchronized Block**
Entire method is synchronized	Only selected section is synchronized
Easy to implement	More control
May lock unnecessary code	Locks only required code
Suitable for complete method protection	Suitable for critical sections

Conclusion

Both methods provide synchronization. A synchronized block is useful when only a small portion of a method needs protection.

What is a Deadlock? How Can It Be Avoided in Multithreaded Programs?

1. Introduction

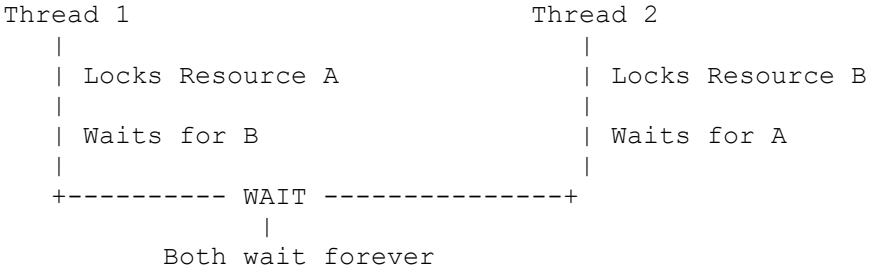
In a multithreaded program, two or more threads may need to access shared resources. If each thread holds a resource and waits for another resource held by another thread, the threads may wait forever.
This situation is called a **deadlock**.

2. Definition of Deadlock

Deadlock is a situation in which two or more threads are permanently blocked because each thread is waiting for a resource or lock held by another thread.
As a result, none of the threads can continue their execution.

3. Simple Example of Deadlock

Consider two threads:



Explanation

- **Thread 1** obtains the lock on Resource A.
- **Thread 2** obtains the lock on Resource B.
- Thread 1 waits for Resource B.
- Thread 2 waits for Resource A.
- Neither thread releases its lock.
- Therefore, both threads remain blocked.

This is called a **deadlock**.

4. Java Example of Deadlock

```
class DeadlockDemo
{
    public static void main(String args[])
    {
        final String resource1 = "Resource A";
        final String resource2 = "Resource B";

        Thread t1 = new Thread()
        {
            public void run()
            {
```

```
                synchronized(resource1)
                {
                    System.out.println("Thread 1 locked Resource A");

                    synchronized(resource2)
                    {
                        System.out.println("Thread 1 locked Resource B");
                    }
                }
            }
        };

        Thread t2 = new Thread()
        {
            public void run()
            {
                synchronized(resource2)
                {
                    System.out.println("Thread 2 locked Resource B");

                    synchronized(resource1)
                    {
                        System.out.println("Thread 2 locked Resource A");
                    }
                }
            }
        };

        t1.start();
        t2.start();
    }
}
```

5. Explanation of the Program

Thread 1

`synchronized(resource1)`

Thread 1 obtains the lock on **Resource A**.

It then tries to obtain the lock on **Resource B**.

Thread 2

`synchronized(resource2)`

Thread 2 obtains the lock on **Resource B**.

It then tries to obtain the lock on **Resource A**.

Result

```
Thread 1 → Holds A → Waiting for B
Thread 2 → Holds B → Waiting for A
```

Both threads wait for each other.

Therefore, a **deadlock occurs**.

6. Conditions for Deadlock

Deadlock generally occurs when the following conditions exist simultaneously:

1. Mutual Exclusion

A resource can be used by only one thread at a time.

2. Hold and Wait
A thread holds one resource while waiting for another resource.
3. No Preemption
A resource cannot be forcibly taken away from a thread holding it.
4. Circular Wait
There is a circular chain of threads where each thread waits for a resource held by the next thread.
Diagram
```
Thread 1
   |
   | waits for
   v
Resource B
   |
   | held by
   v
Thread 2
   |
   | waits for
   v
Resource A
   |
   | held by
   v
Thread 1
```
These conditions together can lead to deadlock.

7. How Can Deadlock Be Avoided?
Deadlock can be avoided or reduced by carefully designing the multithreaded program.
1. Acquire Locks in a Fixed Order
This is one of the most important techniques.
All threads should acquire multiple locks in the **same order**.
Incorrect approach
```
Thread 1 → Lock A → Lock B
Thread 2 → Lock B → Lock A
```
This can cause deadlock.
Better approach
```
Thread 1 → Lock A → Lock B
Thread 2 → Lock A → Lock B
```
Since both threads follow the same order, circular waiting can be avoided.

8. Avoid Unnecessary Locks
Only synchronize the code that actually requires protection.
Example
Instead of synchronizing a complete large method, protect only the critical section:
```
synchronized(obj)
{
    // critical section
```

```
}
```
This reduces unnecessary lock holding.

9. Keep Synchronized Sections Short

A thread should hold a lock for the **shortest possible time**.

Good practice

```
Acquire Lock
     ↓
Perform Required Operation
     ↓
Release Lock
```
Avoid performing lengthy operations while holding a lock.

10. Avoid Nested Locks When Possible

Nested synchronization can increase the possibility of deadlock.

Example

```
synchronized(obj1)
{
    synchronized(obj2)
    {
        // operations
    }
}
```
If another thread locks `obj2` first and then tries to lock `obj1`, a deadlock can occur.
Therefore, unnecessary nested locks should be avoided.

11. Use Proper Thread Coordination

Methods such as:
`wait()`
`notify()`
`notifyAll()`
can be used for proper coordination between threads where appropriate.
They help threads communicate rather than continuously competing for resources.

12. Use Timeout-Based Locking Where Available

In modern Java concurrency APIs, mechanisms such as `tryLock()` with a timeout can be used instead of waiting indefinitely for a lock.
For example, with `ReentrantLock`:

```
if(lock.tryLock())
{
    try
    {
        // critical section
    }
    finally
```

```
    {
        lock.unlock();
    }
}
else
{
    System.out.println("Lock not available");
}
```

This allows a thread to stop waiting indefinitely when a lock cannot be obtained.

13. Important Points for Deadlock Avoidance

1. **Acquire locks in a fixed order.**
2. **Avoid unnecessary locks.**
3. **Keep synchronized sections short.**
4. **Avoid unnecessary nested locks.**
5. **Release locks properly.**
6. **Use proper thread communication.**
7. **Use timeout-based locking where appropriate.**
8. **Design shared-resource access carefully.**

14. Deadlock vs Race Condition

Deadlock	**Race Condition**
Threads wait indefinitely for each other	Threads access shared data concurrently
Program may stop making progress	Program may produce incorrect results
Usually involves locks/resources	Usually involves unsynchronized shared data
Example: Thread 1 waits for B while Thread 2 waits for A	Two threads simultaneously update the same variable
Lock ordering can help avoid it	Synchronization can help prevent it

15. Conclusion

Deadlock is a serious problem in multithreaded programming in which two or more threads wait indefinitely for resources held by one another.

It can be avoided by **using a consistent lock-ordering strategy, minimizing lock usage, keeping synchronized sections short, avoiding unnecessary nested locks, and using suitable thread coordination and lock-management techniques.**